Isaac Sim · Lab 실습

Windows와 Ubuntu 실행 코드

Isaac Sim 6.1 / Isaac Lab 3.0 EA · 8 GB 소형 실습

2026.10.19 · KANT

실습과 결과 파일

실습	실행할 것	확인할 결과
물리	질량이 다른 두 상자 낙하	USD 장면, 높이 CSV
카메라	seed가 다른 장면 촬영	RGB, 깊이 NPY, 메타데이터
보상	거리·시간·충돌에 따른 점수 계산	계산표, 보상 설계 답안
강화학습	Cartpole 무작위 행동·PPO·재생	학습 로그, 체크포인트, 관찰표

설치와 첫 실행을 준비한 뒤 실습을 순서대로 진행한다.

실습 PC와 버전

항목	Windows	Ubuntu
운영체제	Windows 11 ×64 / Python 3.12	Ubuntu 24.04 ×86_64 / Python 3.12
터미널	PowerShell	Bash
Sim / Lab	Sim 6.1.0.0 / Lab v3.0.0-EA	Sim 6.1.0.0 / Lab v3.0.0-EA
공식 최소 예시	RAM 32 GB / VRAM 16 GB	RAM 32 GB / VRAM 16 GB
시험 드라이버	595.97	595.58.03

8 GB는 공식 최소 16 GB 미만이다. 아래 소형 설정도 실행을 보장하지 않으며 PC별 확인이 필요하다.

8 GB 실습 설정과 VM 제한

항목	실습 설정	확인 사항
Sim 화면	640 × 360 / AA 끄기	빈 장면부터 시작
카메라	320 × 240 / 카메라 1개	RGB·깊이 파일 확인
Cartpole	환경 1개 / 작은 과제	안정 실행 후 환경 수 증가
Ubuntu VirtualBox	패키지 설치·CPU 보상 예제	VMSVGA는 NVIDIA RTX가 아님
Ubuntu GPU 실행	NVIDIA GPU를 직접 쓰는 환경	네이티브 설치 또는 적합한 GPU VM

headless도 Isaac Sim의 NVIDIA GPU 요구를 없애지 않는다. VM 설치 완료와 Sim 실행 성공을 구분한다.

실습 ZIP과 작업 폴더

Windows 11 · PowerShell

```
Set-Location C:\isaac-course\windows_lab
Get-ChildItem
Get-ChildItem .\examples
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course/windows_lab"
ls
ls examples
```

examples 폴더와 README.md가 현재 폴더에 보이면 준비 완료

ZIP 안의 windows_lab 폴더를 왼쪽·오른쪽 경로에 각각 놓는다.

기본 도구 준비

Windows 11 · PowerShell

```
# Python 3.12 ×64와 Git을 먼저 설치
# 긴 경로 설정은 관리자 PowerShell
$key = 'HKLM:\SYSTEM\CurrentControlSet'
$key += '\Control\FileSystem'
New-ItemProperty -Path $key `
  -Name LongPathsEnabled -Value 1 `
  -PropertyType DWORD -Force
```

Ubuntu 24.04 · Bash

```
sudo apt update
sudo apt install -y \
  git curl build-essential cmake \
  python3.12-venv python3-pip
```

Windows는 새 일반 PowerShell을 연다. Ubuntu는 설치가 정상 종료되어야 한다.

Windows Python·Git 공식 설치 링크는 발표자 노트와 실행 안내에 있다.

GPU와 Python 확인

Windows 11 · PowerShell

```
nvidia-smi
py -3.12 --version
git --version
```

Ubuntu 24.04 · Bash

```
nvidia-smi
python3.12 --version
git --version
free -h
df -h "$HOME"
```

GPU 이름·VRAM·드라이버와 Python 3.12가 표시되는지 확인

nvidia-smi의 CUDA 표시는 드라이버가 지원하는 수준이다.

uv 설치

Windows 11 · PowerShell

```
$url = 'https://astral.sh/uv/install.ps1'
irm $url | iex
$env:Path = "$env:USERPROFILE" + `
  "\.local\bin;" + $env:Path
uv --version
```

Ubuntu 24.04 · Bash

```
curl -LsSf \
  https://astral.sh/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"
uv --version
```

uv 버전이 출력되면 Lab 설치 도구 준비 완료

명령을 찾지 못하면 새 터미널을 열고 다시 확인한다.

Isaac Sim 전용 Python 환경

Windows 11 · PowerShell

```
$venv = 'C:\isaac-course\simenv'
if (Test-Path $venv) {
  throw '기존 폴더를 먼저 확인하세요.'
}
py -3.12 -m venv $venv
$py = "$venv\Scripts\python.exe"
& $py -m pip install --upgrade pip
```

Ubuntu 24.04 · Bash

```
VENV="$HOME/pa-course/.venv-isaacsim"
test ! -e "$VENV" || exit 1
python3.12 -m venv "$VENV"
PY="$VENV/bin/python"
"$PY" -m pip install --upgrade pip
```

이후 Sim 명령은 Windows의 $py, Ubuntu의 $PY를 사용

이미 설치한 6.1 환경이 있으면 새로 만들지 않고 다음 장의 경로 변수만 설정한다.

Isaac Sim 6.1 패키지 설치

Windows 11 · PowerShell

```
$py = 'C:\isaac-course\simenv' + `
  '\Scripts\python.exe'
$base = 'https://download.pytorch.org'
& $py -m pip install torch==2.11.0 `
  --index-url "$base/whl/cu130"
& $py -m pip install `
  'isaacsim[all,extscache]==6.1.0.0' `
  --extra-index-url https://pypi.nvidia.com
& $py -m pip check
```

Ubuntu 24.04 · Bash

```
VENV="$HOME/pa-course/.venv-isaacsim"
PY="$VENV/bin/python"
BASE='https://download.pytorch.org'
"$PY" -m pip install torch==2.11.0 \
  --index-url "$BASE/whl/cu130"
"$PY" -m pip install \
  'isaacsim[all,extscache]==6.1.0.0' \
  --extra-index-url https://pypi.nvidia.com
"$PY" -m pip check
```

pip check에서 의존성 오류가 없는지 확인한 뒤 다음 단계로 이동

각 명령이 성공한 뒤 다음 명령을 실행한다. CUDA 13을 지원하는 드라이버가 필요하다.

호환성 검사와 첫 실행

Windows 11 · PowerShell

```
$sim = 'C:\isaac-course\simenv' + `
  '\Scripts\isaacsim.exe'
& $sim isaacsim.exp.compatibility_check

# 검사 창을 닫은 뒤 실행
& $sim
```

Ubuntu 24.04 · Bash

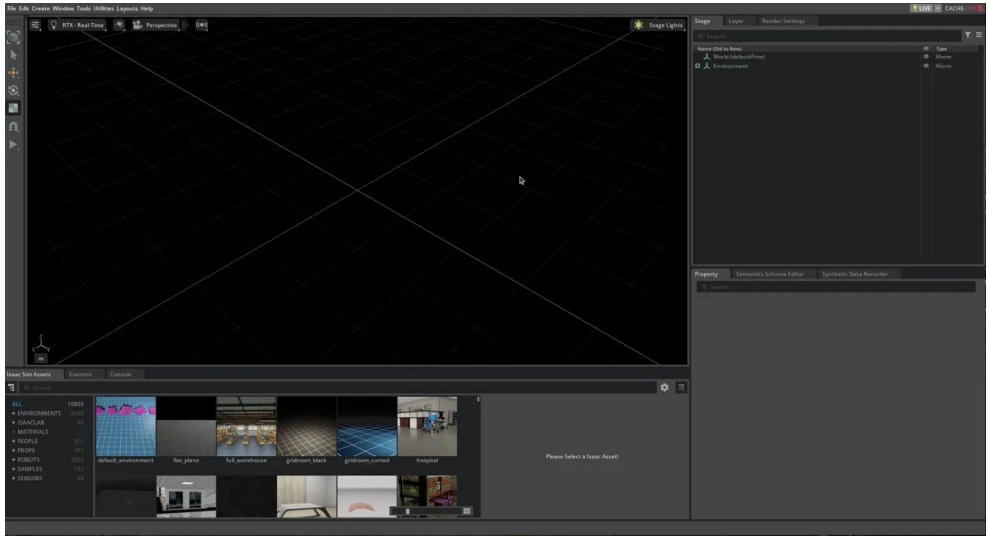
```
VENV="$HOME/pa-course/.venv-isaacsim"
SIM="$VENV/bin/isaacsim"
"$SIM" isaacsim.exp.compatibility_check

# 검사 창을 닫은 뒤 실행
"$SIM"
```

호환성 검사와 기본 화면을 확인한 뒤 Sim을 종료

첫 실행은 약관 확인·확장 다운로드·셰이더 준비가 필요하다.

실습 1 · 상자 낙하



입력 조건

중심 높이 2 m
질량 1 kg과 10 kg

관찰

무거운 상자가 먼저 닿는가?
마지막 중심 높이는 얼마인가?

저장

시작 장면 USD
물리 시각별 높이 CSV

공식 UI 참고 화면. 다음 코드의 실제 실행 결과는 직접 확인한다.

상자 낙하 실행

Windows 11 · PowerShell

```
Set-Location C:\isaac-course\windows_lab
$py = 'C:\isaac-course\simenv' + `
  '\Scripts\python.exe'
& $py .\examples\01_drop_cubes.py
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course/windows_lab"
VENV="$HOME/pa-course/.venv-isaacsim"
PY="$VENV/bin/python"
"$PY" examples/01_drop_cubes.py
```

두 상자가 떨어지고 outputs에 01_drop_실행시각 폴더가 생성

다른 Sim 창을 닫고 실행한다. 기본 600회 앱 업데이트 후 종료한다.

높이와 중력 바꾸기

Windows 11 · PowerShell

```
# 하나씩 실행하고 종료를 기다린다.
& $py .\examples\01_drop_cubes.py `
  --height 4

& $py .\examples\01_drop_cubes.py `
  --gravity 1.62
```

Ubuntu 24.04 · Bash

```
# 하나씩 실행하고 종료를 기다린다.
"$PY" examples/01_drop_cubes.py \
  --height 4

"$PY" examples/01_drop_cubes.py \
  --gravity 1.62
```

기본 조건·높이 4 m·중력 1.62의 접촉 시각을 각각 기록

이전 장의 $py / $PY와 작업 폴더 설정을 유지한다.

높이 CSV와 시작 장면 확인

Windows 11 · PowerShell

explorer .\outputs

Ubuntu 24.04 · Bash

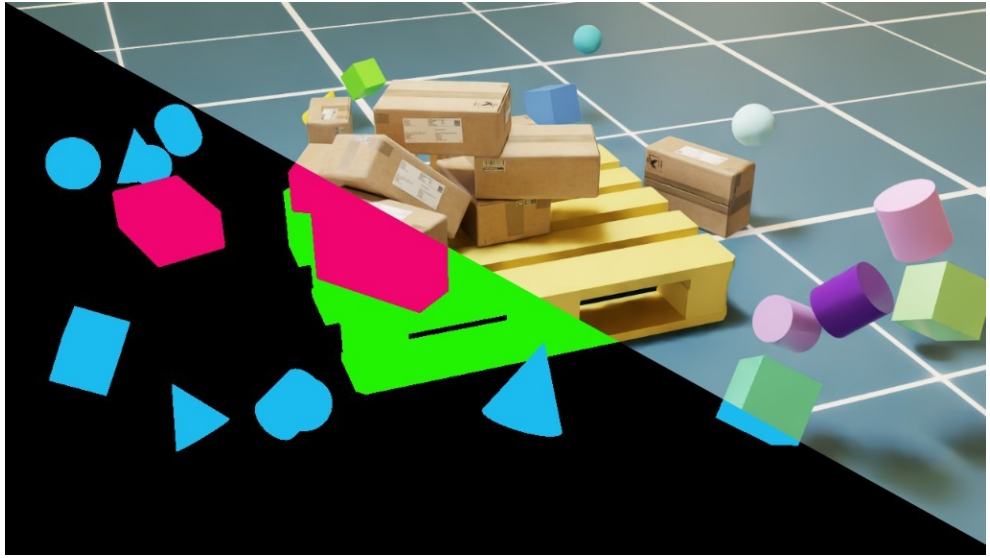
xdg-open ./outputs

질문: 상자가 바닥에 있어도 중심 z가 0이 아닌 이유는?

01_drop 폴더의 heights.csv와 initial_scene.usda를 확인

CSV에서 z ≤ 0.27 m인 첫 행을 접촉의 근사 시각으로 비교한다.

실습 2 · 카메라와 합성 데이터



입력 조건

seed에 따라 색·위치 변경
해상도 320 × 240

데이터

RGB 영상과 깊이 배열
장면 정보를 함께 기록

관찰

seed를 바꿔 비교
카메라 시점도 바꿔보기

공식 데이터 생성 참고 화면. 이번 예제는 상자 장면을 촬영한다.

카메라 촬영 두 번 실행

Windows 11 · PowerShell

```
& $py .\examples\02_camera_capture.py `
  --seed 7

& $py .\examples\02_camera_capture.py `
  --seed 42

explorer .\outputs
```

Ubuntu 24.04 · Bash

```
"$PY" examples/02_camera_capture.py \
  --seed 7

"$PY" examples/02_camera_capture.py \
  --seed 42

xdg-open ./outputs
```

02_camera 폴더 두 개의 RGB에서 상자 위치와 색을 비교

두 실행을 순서대로 진행한다. 카메라 실습에도 RTX GPU가 필요하다.

카메라 출력 파일

파일	내용	확인할 것
rgb.png	렌더링한 색 영상	상자·바닥이 보이는가
depth_m.npy	깊이 원본 배열	단위 m, 이미지 평면까지 거리
depth_preview.png	0~5 m 보기용 영상	실제 거리 분석에는 NPY 사용
scene_metadata.json	seed·색·위치·해상도	영상과 같은 실행 폴더인가
scene.usda	카메라를 포함한 장면	시점을 바꿔 다시 확인

320 × 240과 깊이 단위를 기록한다. 이 실습은 데이터를 생성하며 VLM 모델은 실행하지 않는다.

실습 3 · 보상 계산

Windows 11 · PowerShell

```
Set-Location C:\isaac-course\windows_lab
py -3.12 .\examples\03_reward_design.py
Get-Content .\outputs\reward_examples.csv
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course/windows_lab"
python3.12 examples/03_reward_design.py
cat outputs/reward_examples.csv
```

Self-check passed와 사례별 보상 표가 출력

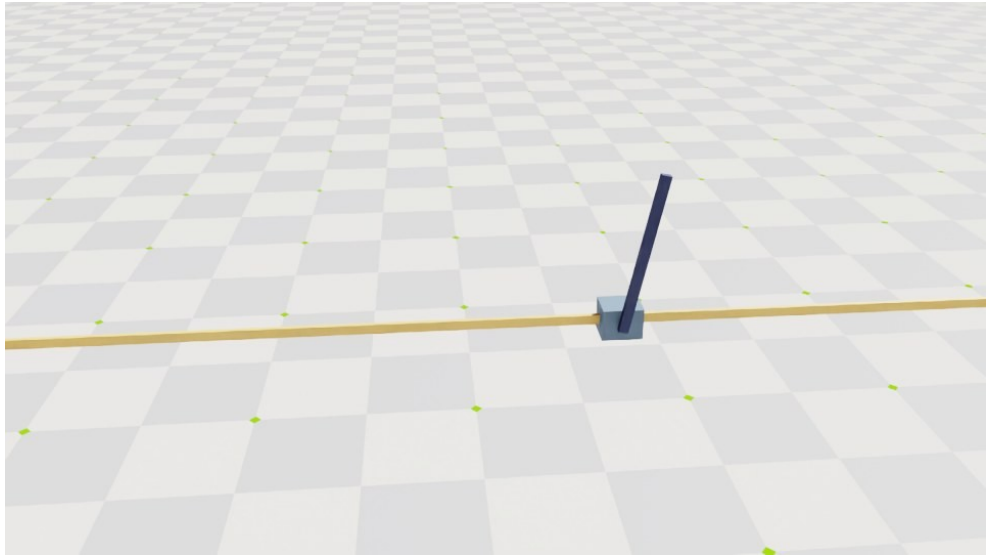
이 예제는 Python만 사용한다. GPU·Sim 설치 없이도 실행할 수 있다.

보상 값과 편법 행동

상황	보상	생각할 문제
0.1초 동안 0.2 m 접근	0.399	계산식에서 거리 항은 얼마인가
0.1초 동안 0.2 m 후퇴	−0.401	왕복하면 점수가 남는가
0.1초 동안 정지	−0.001	시간 비용이 없다면 어떤가
0.2 m 접근하며 도착	10.399	도착 보상을 계속 주면 어떤가
0.2 m 접근하며 충돌	−19.601	충돌 벌점이 너무 작다면?

보상 = 2 × (이전 거리 − 현재 거리) − 0.01 × dt + 도착 또는 충돌 항

실습 4 · Cartpole 정책 학습



관측과 행동

수레·막대의 위치와 속도
수레에 가하는 힘

학습

RSL-RL의 PPO
환경 1개부터 실행

비교

무작위 행동
학습한 정책 재생

공식 과제 참고 화면. 수업의 실행 화면은 설치한 EA 버전에서 확인한다.

Isaac Lab 고정 버전 받기

Windows 11 · PowerShell

```
Set-Location C:\isaac-course
$repo = 'https://github.com/isaac-sim/'
$repo += 'IsaacLab.git'
git clone --depth 1 `
  --branch v3.0.0-EA `
  $repo IsaacLab-6.1
Set-Location .\IsaacLab-6.1
git rev-parse HEAD
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course"
REPO='https://github.com/isaac-sim/'
REPO+="IsaacLab.git"
git clone --depth 1 \
  --branch v3.0.0-EA \
  "$REPO" IsaacLab-6.1
cd IsaacLab-6.1
git rev-parse HEAD
```

v3.0.0-EA 저장소를 별도 폴더에 받는다.

폴더가 이미 있거나 clone이 실패하면 멈추고 기존 버전부터 확인한다.

커밋 확인과 Lab 환경 설치

Windows 11 · PowerShell

```
$pin = 'ae37b028ea415c91ea2b'
$pin += 'c32609efcd759ed2b974'
if ((git rev-parse HEAD) -ne $pin) {
  throw '수업 기준 커밋과 다릅니다.'
}
uv sync --frozen --extra isaacsim
```

Ubuntu 24.04 · Bash

```
PIN='ae37b028ea415c91ea2b'
PIN+='c32609efcd759ed2b974'
test "$(git rev-parse HEAD)" = "$PIN" \
  || exit 1
uv sync --frozen --extra isaacsim
```

커밋 일치 후 uv sync가 오류 없이 종료되어야 한다.

Lab 폴더의 .venv에 Sim 6.1을 별도로 설치한다. uv.lock을 갱신하지 않는다.

Lab의 Sim 버전과 과제 확인

Windows 11 · PowerShell

```
Set-Location C:\isaac-course\IsaacLab-6.1
$check = 'from importlib.metadata '
$check += 'import version; '
$check += "print(version('isaacsim'))"
uv run --frozen --extra isaacsim `
  python -c $check
uv run --frozen --extra isaacsim `
  isaaclab list_envs --show_presets
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course/IsaacLab-6.1"
CHECK='from importlib.metadata '
CHECK+='import version; '
CHECK+="print(version('isaacsim'))"
uv run --frozen --extra isaacsim \
  python -c "$CHECK"
uv run --frozen --extra isaacsim \
  isaaclab list_envs --show_presets
```

버전 6.1.0.0과 Isaac-Cartpole 과제를 확인

목록 조회는 환경 점검이다. 실제 물리·학습은 다음 단계에서 확인한다.

Cartpole 무작위 행동

Windows 11 · PowerShell

```
uv run --frozen --extra isaacsim `
  isaaclab random_agent `
  --task Isaac-Cartpole `
  --num_envs 1 `
  physics=isaacsim_physx --viz kit
```

Ubuntu 24.04 · Bash

```
uv run --frozen --extra isaacsim \
  isaaclab random_agent \
  --task Isaac-Cartpole \
  --num_envs 1 \
  physics=isaacsim_physx --viz kit
```

수레·막대 1개의 움직임과 초기화를 관찰

학습된 정책이 아니다. 창을 닫거나 Ctrl+C로 종료한 뒤 다음 단계로 이동한다.

PPO 학습 실행

Windows 11 · PowerShell

```
uv run --frozen --extra isaacsim `
  isaaclab train `
  --rl_library rsl_rl `
  --task Isaac-Cartpole `
  --num_envs 1 --seed 42 `
  --max_iterations 300 `
  physics=isaacsim_physx --viz kit
```

Ubuntu 24.04 · Bash

```
uv run --frozen --extra isaacsim \
  isaaclab train \
  --rl_library rsl_rl \
  --task Isaac-Cartpole \
  --num_envs 1 --seed 42 \
  --max_iterations 300 \
  physics=isaacsim_physx --viz kit
```

logs 아래 학습 로그와 체크포인트가 저장되는지 확인

300은 갱신 횟수다. 수렴·균형 성공을 보장하지 않으므로 실제 결과를 기록한다.

학습한 정책 재생

Windows 11 · PowerShell

```
uv run --frozen --extra isaacsim `
  isaaclab play `
  --rl_library rsl_rl `
  --task Isaac-Cartpole `
  --num_envs 1 --seed 123 `
  --checkpoint latest `
  physics=isaacsim_physx --viz kit
```

Ubuntu 24.04 · Bash

```
uv run --frozen --extra isaacsim \
  isaaclab play \
  --rl_library rsl_rl \
  --task Isaac-Cartpole \
  --num_envs 1 --seed 123 \
  --checkpoint latest \
  physics=isaacsim_physx --viz kit
```

학습 전과 균형 유지·초기화·카트 위치를 비교

latest는 호환되는 최근 실행을 선택한다. 실제 체크포인트 경로를 따로 기록한다.

화면 없이 학습

Windows 11 · PowerShell

```
uv run --frozen --extra isaacsim `
  isaaclab train `
  --rl_library rsl_rl `
  --task Isaac-Cartpole `
  --num_envs 1 --seed 42 `
  --max_iterations 300 `
  physics=isaacsim_physx --viz none
```

Ubuntu 24.04 · Bash

```
uv run --frozen --extra isaacsim \
  isaaclab train \
  --rl_library rsl_rl \
  --task Isaac-Cartpole \
  --num_envs 1 --seed 42 \
  --max_iterations 300 \
  physics=isaacsim_physx --viz none
```

창 없이 학습 로그가 진행되는지 확인

none은 화면 생략이다. GPU는 필요하며 환경 수를 줄이면 학습 조건도 달라진다.

학습 전후 관찰 기록

항목	무작위 행동	학습 후 정책
균형 유지 경향	직접 기록	직접 기록
실패·초기화	시간 제한과 실패 구분	같은 관찰 범위로 비교
카트 위치	끝으로 치우치는지	끝으로 치우치는지
실행 조건	환경 수·seed·GPU	갱신 횟수·체크포인트

성과가 개선되지 않아도 그대로 기록한다. 보상 상승만으로 실물 배포 가능성을 판단하지 않는다.

실행이 안 될 때

증상	Windows 확인	Ubuntu 확인
Sim 모듈을 찾지 못함	$py가 simenv Python인가	$PY가 Sim 전용 Python인가
줄 연결 문법 오류	백틱 뒤 공백이 없는가	역슬래시 뒤 공백이 없는가
GPU·RTX 오류	nvidia-smi, 지원 드라이버	nvidia-smi, 데스크톱 세션
Lab 과제·인자 오류	EA 커밋·작업 폴더 확인	EA 커밋·작업 폴더 확인
체크포인트 없음	Train 로그와 실제 파일 확인	Train 로그와 실제 파일 확인

터미널에서 가장 먼저 발생한 오류부터 확인한다. 설치 명령을 반복 실행하기 전에 원인을 확인한다.

배포 예제 자체 점검

Windows 11 · PowerShell

```
Set-Location C:\isaac-course\windows_lab
py -3.12 .\99_self_check.py
```

Ubuntu 24.04 · Bash

```
cd "$HOME/pa-course/windows_lab"
python3.12 99_self_check.py
```

Python 예제 컴파일과 보상 계산 점검이 PASS인지 확인

이 점검은 Sim 렌더링·물리·Lab 학습의 성공을 판정하지 않는다.

실습 결과 정리

실습	보관할 파일	함께 기록할 조건
물리	heights.csv / initial_scene.usda	높이·중력·접촉 시각
카메라	RGB 2장 / 깊이 / 메타데이터	seed·해상도·카메라 시점
보상	reward_examples.csv / 답안	점수 계산·편법 행동 설명
강화학습	로그 / 체크포인트 / 관찰표	버전·커밋·GPU·seed·환경 수

실행 명령은 실습 ZIP의 RUN_COMMANDS.md, 문제와 기록 양식은 EXERCISES.md에 있다.

부록 · 기존 Workstation 설치 사용

Windows 11 · PowerShell

```
# 아래 경로를 실제 6.1 설치 위치로 변경
Set-Location C:\isaac-course\windows_lab
& 'C:\isaacsim\python.bat' `
  .\examples\01_drop_cubes.py
```

Ubuntu 24.04 · Bash

```
# 아래 경로를 실제 6.1 설치 위치로 변경
cd "$HOME/pa-course/windows_lab"
"$HOME/isaacsim/python.sh" \
  examples/01_drop_cubes.py
```

기존 Sim 6.1의 Python 런처로 같은 예제를 실행

경로는 예시다. Lab은 앞서 안내한 별도 uv 환경을 그대로 사용한다.